Week10 – Serial Communication, Line Follower Robot (2)
PC ____ รหัส _____ ชื่อ-สกุล _____

1/8: การทดลอง

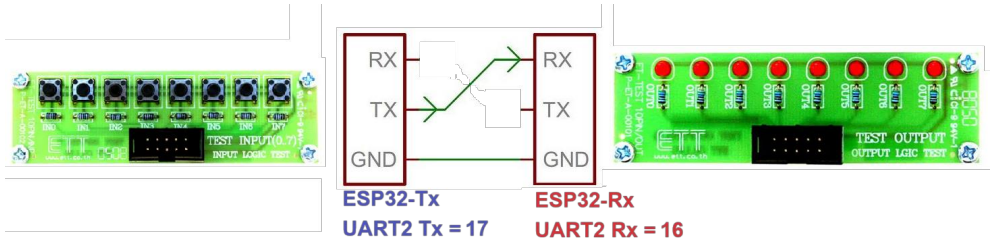
1. MCS51-Tx: จากโปรแกรม “Lab10Srl1.Asm” ปรับให้วนลูปส่ง “รหัสนักศึกษา, ชื่อ-นามสกุล, ขึ้นบรรทัดใหม่-ซิดซ้าย, หน่วงเวลา 500mSec” จำนวน 10 บรรทัด แล้วขึ้นข้อความ “I Want A” ในบรรทัดที่ 11

2. MCS51-Rx:
   - 2.1 จากโปรแกรม “Lab10Srl2.Asm” เดิมมีการรับคีย์เฉพาะ A หรือ B ให้ปรับปรุงเพื่อแสดงผลเฉพาะกรณีที่มีการกดคีย์จาก 0-9 และคีย์อื่นๆ ให้แสดง – (ขีดกลาง)
   - 2.2 ถ้ารับค่าคีย์เป็น 1-9 ให้ LED P3.7 กระพริบตามจำนวนที่ป้อนแล้วดับ, ถ้าคีย์อื่นๆ ให้ LED P3.2 ดับ
   - 2.3 ถ้ารับค่าคีย์เป็น 0, +, - หากเป็น {+ , 0 , - } ให้ 7Segment { นับขึ้น คงค่าการนับ นับลง}

3. MCS51-Tx, Rx:
   - ทดสอบส่งค่าผ่าน Serial-1 ด้วยค่าการกดสวิตซ์ SW0 Sw1 Sw2 Sw3 หากกด SW0 ให้ส่ง ASCII ของเลข 0, กด Sw1 = “1”, กด Sw2 = “2” และ กด Sw3 = “3”
   - ทดสอบรับค่าจาก Serial-1 หากรับ ASCII เลข 0-9 ให้แสดงเขข 0-9 ที่ 7Segment
   - ต่อสาย Tx จากบอร์ด1 ไปยัง Rx ของบอร์ด2 และ GND-GND และทดสอบการทำงาน

4. ESP32-Rx: เขียนโปรแกรมสร้างสัญญาน โดยรับคำสั่งเป็น Serial Command จำนวน 4 คำสั่ง คือ P=xxx<Cr>, D=yy<Cr>, G, ?

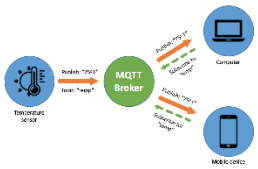
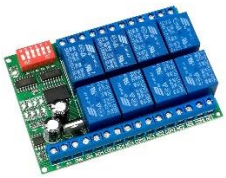
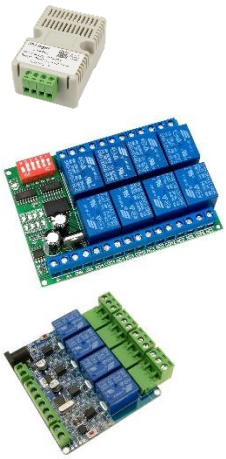
5. ESP32-Tx, Rx: ให้นักศึกษากับคู่ระหว่างเครื่องเลขคี่ กับ เลขคี่+1 ทำการเขียนโปรแกรมร่วมกัน ดังนี้
   - PC-คู่ ให้เป็นคนส่ง: ต่อวงจรตาม “Digital Input – 8 LED”, ใช้ Serial2 ในการรับส่งข้อมูล
   - PC-คี่ ให้เป็นคนรับ: ต่อวงจรตาม “Digital Output – 8 LED”, ใช้ Serial2 ในการรับส่งข้อมูล



1.MCS51 Tx	2.MCS51 Rx	3.MCS51 Tx-Rx	4.ESP32 Rx	5.ESP32 Tx + Rx	6.ESP32 OTA

6. ESP32 OTA: ทดลอง Upload Code สลับกันไปมาระหว่าง Blink01 กับ Blink02 ด้วยวิธีการทั้ง 4 วิธี
    - 5.1 Test31 - Arduino Basic OTA
    - 5.2 Test32 – Arduino Web Updater OTA
    - 5.3 Test33 – Arduino IAP (In Application Programing)
    - 5.4 Test33 – Blynk OTA

7. Modbus: Industrial Serial Protocol
    - 6.1 ETT-RS485 v2 Board - Input/Output and MQTT
    - 6.2 ETT-RS485 v2 Board – Blynk2.0
    - 6.3 RS485 MODBUS/RTU Read Modbus Device
    - 6.4 RS485 MODBUS/RTU Write Modbus Device
    - 6.5 RS485 MODBUS/RTU Read/Write Device
    - 6.6 Blynk2.0 to MODBUS/RTU Device
    - 6.7 MODBUS/TCP Read/Write Device

IoT with MQTT Lens		Blynk 2.0	
			Blynk 2.0
Read Modbus Device		**Write Modbus Device**	
			
Read/Write Modbus Device		**Blynk 2.0 ↔ Modbus Device**	
		Blynk 2.0	

2/8: Serial Communication
00-Serial Communication 01-OSI Model and Industrial Communication Protocol 02-Asynchronous RS232 RS422 RS485

< Insert >

3/8: MCS51 Serial Communication

1. รีจีสเตอร์ที่เกี่ยวข้อง

TMOD	**G(T1)**	**C/T**	**M1**	**M0**	**G(T0)**	**C/T**	**M1**	**M0**
TCON	**TF1**	**TR1**	**TF0**	**TR0**	**IE1**	**IT1**	**IE0**	**IT0**
SCON	**SM0**	**SM1**	**SM2**	**REN**	**TB8**	**RB8**	**TI**	**RI**
PCON	**SMOD**	**-**	**-**	**-**	**GF1**	**GF0**	**PD**	**IDL**
T2CON	**TF2**	**EXF2**	**RCLK**	**TCLK**	**EXEN2**	**TR2**	**C/T2**	**CP/RL2**

https://studylib.net/doc/13957472/8051-family-special-function-registers--pcon-%E2%80%93-power-cont...

PCON – Power Control Register

D7	D6	D5	D4	D3	D2	D1	D0
SMOD	x	x	x	GF1	GF0	PD	IDL

Address: 87H (not bit addressable)

SMOD – Serial mode bit used to determine the baud rate with Timer 1.

$$\text{Baud rate} = \frac{\text{Oscillator frequency in Hz}}{N[256-(TH1)]}$$

If SMOD = 0 then N = 384. If SMOD = 1 then N = 192. TH1 is the high byte of timer 1 when it is in 8-bit autoreload mode.

GF1 and GF0 are General purpose flags not implemented on the standard device
PD is the power down bit. Not implemented on the standard device
IDL activate the idle mode to save power. Not implemented on the standard device

TCON – Timer Control Register

D7	D6	D5	D4	D3	D2	D1	D0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

Address: 88H (bit addressable)
TF1 – Timer 1 overflow flag
TR1 – Timer 1 run control bit
TF0 – Timer 0 overflow flag
TR0 – Timer 0 run control bit
IE1 – External interrupt 1 edge flag. Set to 1 when edge detected.
IT1 – Edge control bit for external interrupt 1. 1 = edge, 0 = level
IE0 – External interrupt 0 edge flag. Set to 1 when edge detectd
IT0 – Edge control bit for external interrupt 0. 1 = edge, 0 = level

SCON – Serial Control Register

D7	D6	D5	D4	D3	D2	D1	D0
SM0	SM1	SM2	REN	TB8	RB8	TI	RI

Address: 98H (bit-addressable)

SM0	**SM1**	**Operation**	**Baud rate**
0	0	Shift register	Osc/12
0	1	8-bit UART	Set by timer
1	0	9-bit UART	Osc/12 or Osc/64
1	1	9-bit UART	Set by timer

SM2 – Enables multiprocessor communication in modes 2 and 3.
REN – Receiver enable
TB8 – Transmit bit 8. This is the 9th bit transmitted in modes 2 and 3.
RB8 – Receive bit 8. This is the 9th bit received in modes 2 and 3.
TI – Transmit interrupt flag. Set at end of character transmission. Cleared in software.
RI – Receive interrupt flag. Set at end of character reception. Cleared in software.

TMOD – Timer Mode Control Register

D7	D6	D5	D4	D3	D2	D1	D0
Gate	$C/\overline{T}$	M1	M0	Gate	$C/\overline{T}$	M1	M0
Timer 1				**Timer 0**			

Gate – if 1 timer x is enabled when intx is high and TRx is high. if 0 timer x is enabled when TRx is high.

$C/\overline{T}$ - if 1 timer x is clocked from Tx pin. if 0 timer x is clocked from oscillator/12

M1	**M0**	**Mode**
0	0	13-bit mode for compatibility to 8048 family
0	1	16- bit Timer/Counter. User must reload in software
1	0	8-bit autoreload. TLx is automatically reloaded from THx
1	1	TL0 is 8-bit counter controlled by Timer0 control bits. TH0 is 8-bit counter controlled by Timer1 control bits. Timer 1 is stopped

T2CON – Timer 2 Control Register

D7	D6	D5	D4	D3	D2	D1	D0
TF2	EXF2	RCLK	TCLK	EXEN2	TR2	$C/\overline{T2}$	$CP/\overline{RL2}$

Address: 0C8H (bit addressable)
TF2 – Timer 2 overflow flag
EXF2 – Timer 2 external flag.
RCLK – Receive clock. When set causes the serial port to use timer 2 for reception.
TCLK – Transmit clock. When set causes the serial port to use timer 2 for transmission.
$C/\overline{T2}$ – Counter/Timer select. if 0 use internal timer. if 1 use external pin
$CP/\overline{RL2}$ –Capture/reload flag.

IE – Interrupt Enable Register

D7	D6	D5	D4	D3	D2	D1	D0
EA	x	ET2	ES	ET1	EX1	ET0	EX0

Address: 0A8H (bit addressable)
EA – Global interrupt enable
x – not defined
ET2 – Timer 2 interrupt enable
ES – Serial port interrupt enable
ET1 – Timer 1 interrupt enable
EX1 – External interrupt 1 enable
ET0 – Timer 0 interrupt enable
EX0 – External interrupt 0 enable

IP – Interrupt Priority Register

D7	D6	D5	D4	D3	D2	D1	D0
x	x	PT2	PS	PT1	PX1	PT0	PX0

Address: 0B8H (bit addressable)
x – not defined
PT2 – Priority for timer 2 interrupt
PS – Priority for serial port interrupt
PT1 – Priority for timer 1 interrupt
PX1 – Proiority for external interrupt 1
PT0 – Priority for timer 0 interrupt
PX0 – Priority for external interrupt 0

2. เริ่มต้นใช้งาน Serial Communication

a. Data Frame → SCON

b. Baud Rate → TMOD, TCON, PMOD, T2CON

c. Interrupts → IE, IP

Using Timer1, Mode2 *http://www.intel.com/design/mcs51/applnots/270490.htm*

Serial Mode1,3 และใช้ Timer 1Mode2 →

$$\text{BaudRate} = \frac{2^{SMOD1} \times \text{Oscillater Frequency}}{32 \times 12 \times [256 - (th1)]}$$

9600 @11.0592MHz → SMOD1=0, TH1=253(FDH)
9600 @18.432MHz → SMOD1=0, TH1=251(FBH) (X1 Mode)
19200 @18.432MHz → SMOD1=0, TH1=251(FBH) (X2 Mode)

```
MOV   SCON,#050H  ; Serial Port Mode 3
ANL   PCON,#7FH   ; SMOD 1=2
MOV   TMOD,#20H   ; Timer1 Mode2
MOV   TH1,#0FDH   ; Reload value for desired baud
SETB  TR1         ; Turn on Timer1
```

Using Timer2

$$\text{Buad rate} = \frac{\text{Oscillater Frequency}}{32 \times [65536 - (RCAP2H,RCAP2L)]}$$

9600 @11.0592MHz RCAP2H,RCAP2L = 65500 = FFDCH
9600 @18.432MHz RCAP2H,RCAP2L = 65476 = FFC4H (X1 Mode)
19200 @18.432MHz RCAP2H,RCAP2L = 65476 = FFC4H (X2 Mode)

```
MOV   SCON,#01010000B   ; Mode1 Serial Port
MOV   RCAP2H,#0FFH      ; Reload values for desired
MOV   RCAP2L,#0DCH      ; baud rate 9600@11.0592
MOV   T2CON,#00110100B  ; Timer 2 as baud rate generator
                        ; and turn on Timer 2
```

```
T2CON EQU  0C8H
RCAP2L     EQU  0CAH
RCAP2H     EQU  0CBH
TL2        EQU  0CCH
TH2        EQU  0CDH
TR2        EQU  T2CON.2
TF2        EQU  T2CON.7
ET2        EQU  IE.5
```

การส่งค่าแบบ Polling

```
        MOV     SBUF,A      ; Send Data
        JNB     TI,$        ; Wait Until Tx Complete
        CLR     TI          ; Clear Tx Flag
```

การรับค่าแบบ Polling

```
        CLR     RI          ; Clear Rx Flag
        JNB     RI,$        ; Wait Until Rx Complete
        MOV     A,SBUF      ; Save Data
```

การรับค่าแบบ Interrupts

```
        SETB    ES          ; Enable Serial Interrupts
        SETB    EA          ; Enable All


        ORG     0023H       ; Serial Interrupt Vector 0023H
        JMP     Srl_ISR     ; Jump to ISR


Srl_ISR:CLR     ES          ; Disable Serial Interrupts
        JB      TI,_X_Srl   ; If Tx Exit
        MOV     A,SBUF      ; Save Data
        MOV     SBUF,A      ; Echo Data
        MOV     Rx_Data,A   ; Save to Variable
_X_Srl: CLR     TI          ; Clear Ti Flag
        CLR     RI          ; Clear Ri Flag
        SETB    ES          ; Enable Serial Interrupts
        RETI                ; Return from Interrupts
```

3 การทดลอง

1. ศึกษาการเขียนโปรแกรมเพื่อส่งค่าผ่าน Serial Port เพื่อแสดงผลที่หน้าจอโปรแกรม Hyper Terminal จากโปรแกรม “Lab10Srl1.Asm” และทดสอบการทำงานของโปรแกรม
   - ทดสอบการทำงานของโปรแกรมของโปรแกรม “Lab10Srl1.Asm”

   1.1 จากโปรแกรม “Lab10Srl1.Asm” ปรับให้บอร์ดทำหน้าที่ส่งข้อมูลไปยังเครื่องพีซี ด้วย Baud Rate = 9600 bps ให้โปรแกรมวนลูปส่งรหัสแอสกี้ออกไปยังเครื่องพีซีทีละ 1 ตัวทุก ๆ 0.5 วินาที(โดยประมาณ) ➔ “ 0,1,2,3,4,5,6,7,8,9,ขึ้นบรรทัดใหม่ และชิดซ้ายบรรทัด”

   1.2 จากโปรแกรม “Lab10Srl1.Asm” ปรับให้วนลูปส่ง “รหัสนักศึกษา, ชื่อ-นามสกุล, ขึ้นบรรทัดใหม่-ชิดซ้าย, หน่วงเวลา 500mSec” จำนวน 10 บรรทัด แล้วขึ้นข้อความ “I Want A” ในบรรทัดที่ 11

2. ศึกษาการเขียนโปรแกรมเพื่อรับค่าจาก Serial Port ผ่านโปรแกรม Hyper Terminal จากโปรแกรม “Lab10Srl2.Asm” และทดสอบการทำงานของโปรแกรม โดยผลการทดสอบเมื่อกดค่า A หรือ B ที่ PC Keyboard ไมโครคอนโทร์ลเลอร์จะแสดงค่าที่ 7Segment Display
   - ทดสอบการทำงานของโปรแกรม “Lab10Srl2.Asm”
   - จากโปรแกรม “Lab10Srl2.Asm” เดิมมีการรับคีย์เฉพาะ A หรือ B ให้ปรับปรุงเพื่อแสดงผลเฉพาะกรณีที่มีการกดคีย์จาก 0-9 และคีย์อื่นๆ ให้แสดง – (ขีดกลาง)
   - จากโปรแกรม “Lab10Srl2.Asm” ถ้ารับค่าคีย์ 0, +, - หากเป็น + ให้ 7Segmentนับขึ้น, หากเป็น 0 ให้คงค่าการนับ และหากเป็น - ให้นับลง

3. การสร้างโปรแกรมย่อยเพื่อเรียกใช้
   - พิมพ์โค้ดโปรแกรมตามกรอบ “mySerial.A51” แล้วทำการบันทึกเป็น “mySerial.A51”
   - พิมพ์โค้ดโปรแกรมตามกรอบ “myDelay.A51” แล้วทำการบันทึกเป็น “myDelay.A51”
   - โปรแกรมหลักสามารถเรียกใช้ทุกชุดคำสั่งย่อยที่มีใน mySerial.A51 และ myDelay.A51ได้ทุกชุดแต่ต้องมีการประกาศบรรทัด $Include (xxx.A51) ที่โปรแกรมหลักก่อน
   - ทดสอบการทำงานของโปรแกรม “Lab10Srl3.Asm”
   - จากโปรแกรม “Lab10Srl3.Asm” ปรับปรุงให้การนับ Counter เป็นแบบ BCD

4. การรับค่าแบบ Serial Interrupts
   - ทดสอบการทำงาน “Lab10Srl5.Asm”
   - ปรับโปรแกรมให้รับค่าจาก Serial Terminal เพื่อคุม LED P3.7
     - ถ้ารับค่าคีย์เป็น 1-9 ให้ LED P3.7 กระพริบตามจำนวนที่ป้อนแล้วดับ
     - ถ้าคีย์อื่นๆ ให้ LED P3.2 ดับ

5. การใช้งานด้วย Ride-C51
   - ทดสอบการทำงาน “Lab10Srl4.C”
   - ปรับโปรแกรม Ride-C51 ให้รับค่าผ่าน Serial Terminal แล้วแสดงที่ 7Segment

6. คำถามท้ายการทดลอง

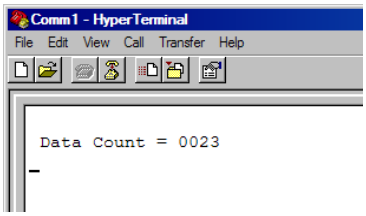
1. จงเขียนโปรแกรมเพื่อรับอินพุตจากสวิตช์ที่ต่อกับพอร์ต P2 จำนวนทั้งสิ้น 8 ตัว โดยกำหนดให้ P2.1 – P2.7 ทำหน้าที่กำหนดจำนวนบรรทัดของข้อมูลทั้งหมดที่จะส่งไปยังโปรแกรมไฮเปอร์เทอร์มินอล ดังนี้

พอร์ต	การทำงาน
P2.0 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบแบบไม่รู้จบ
P2.1 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 1 บรรทัด
P2.2 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 2 บรรทัด
P2.3 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 3 บรรทัด
P2.4 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 4 บรรทัด
P2.5 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 5 บรรทัด
P2.6 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 6 บรรทัด
P2.7 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 7 บรรทัด

2. ให้เขียนโปรแกรมสำหรับนับจำนวนชิ้นงานที่ไหลผ่านสายพานลำเลียง**ข้อสอบเก่า (แนวข้อสอบ)**

Step 1: ทดสอบ Serial Port โดยให้ส่งข้อมูลไปยังเครื่องพีซี ด้วย Baud Rate = 9600 bps ให้โปรแกรมวนลูปส่งรหัสแอสกี้ออกไปยังเครื่องพีซีทีละ 1 ตัวทุก ๆ 0.5 วินาที ตามลำดับดังนี้ “0,1,2,3,4,5,6,7,8,9, ขึ้นบรรทัดใหม่ และชิดซ้ายบรรทัด"

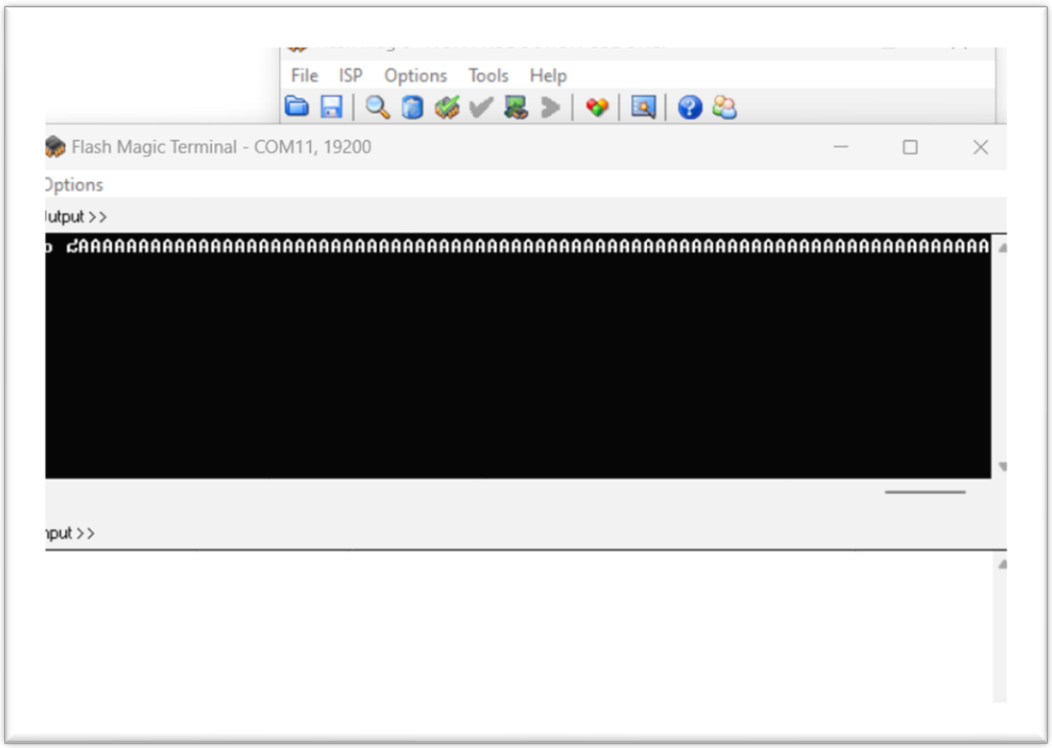
Step 2: ทดสอบการนับโดยให้โปรแกรมควบคุม Timer 0 ให้ทำงานเป็น Counter mode 1 ใช้นับเลข HEX จากสัญญาณภายนอกจากนั้นนำค่า THOTL0 ส่งออกผ่าน Serial Port ไปยังเครื่องพีซีเพื่อแสดงผล



5 โปรแกรมทดสอบ

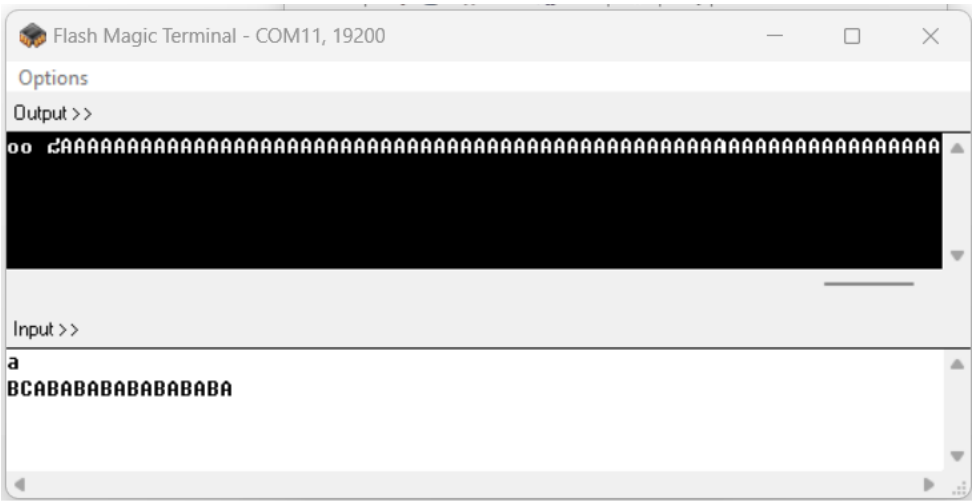
; Lab10Srl1.ASM >> Transmit Data

```
                ORG     0000H                   ; Lab10Srl1.Sub
                JMP     0100H

                ORG     0100H
                MOV      SP,#3FH
                CLR     EA                      ; Disable All
                SETB    P1.0                    ; Input Pin
                JB      P1.0,$                  ; Wait Press P1.0
                CALL    Delay                   ; Start Delay
                MOV     TMOD,#00100001B         ; T1M2 T0M1
                MOV     SCON,#01010000B         ; Serial Mode1
                MOV     TH1,#0FBH               ; 9600 FBH@18.432MHz
                SETB    TR1                     ; Start Timer1

Main_Loop:      MOV     A,#'A'
                MOV     SBUF,A
                JNB     TI,$
                CLR     TI
                CALL    Delay
                JMP     Main_Loop

Delay:          CLR     A
                MOV     B,A
Dly00:          DJNZ    Acc,Dly00
                DJNZ    B,Dly00
                RET

                END
```



; Lab10Srl2.ASM Polling Receive Data

```
T2CON           EQU     0C8H
RCAP2L          EQU     0CAH
RCAP2H          EQU     0CBH

                ORG     0000H                   ; Lab10Srl2.Sub
                JMP     0100H

                ORG     0100H
                MOV     SP,#3FH
                CLR     EA                      ; Disable All
                MOV     SCON,#01010000B         ; Mode1 Serial Port
                MOV     RCAP2H,#0FFH            ; Reload values for desired
                MOV     RCAP2L,#0C4H            ; baud rate 9600@11.0592
                MOV     T2CON,#00110100B        ; Timer 2 as baud rate generator
                                                ; and turn on Timer 2

Main_Loop:      CLR     RI                      ; Ready for New Data
                JNB     RI,$                    ; Wait Until Data In
                MOV     A,SBUF                  ; Get Data In

_IS_ASCII_A:    CJNE    A,#'A',_IS_ASCII_B
                MOV     A,#0AH
                CALL    Display
                JMP     Main_Loop

_IS_ASCII_B:    CJNE    A,#'B',_IS_ASCII_Etc
                MOV     A,#0BH
                CALL    Display
                JMP     Main_Loop

_IS_ASCII_Etc:  NOP
                JMP     Main_Loop

Display:
                MOV     DPTR,#Seg_Tab
                MOVC    A,@A+DPTR
                CPL     A
                MOV     P0,A
                CLR     P1.4
                CLR     P1.5
                RET

Seg_Tab:        DB      77H, 11H, 6DH, 5DH, 1BH, 5EH, 7EH, 15H
                DB      7FH, 5FH, 3FH, 7AH, 66H, 79H, 6EH, 2EH

                END
```



; 1/3 Edit and Save to "mySerial.A51"

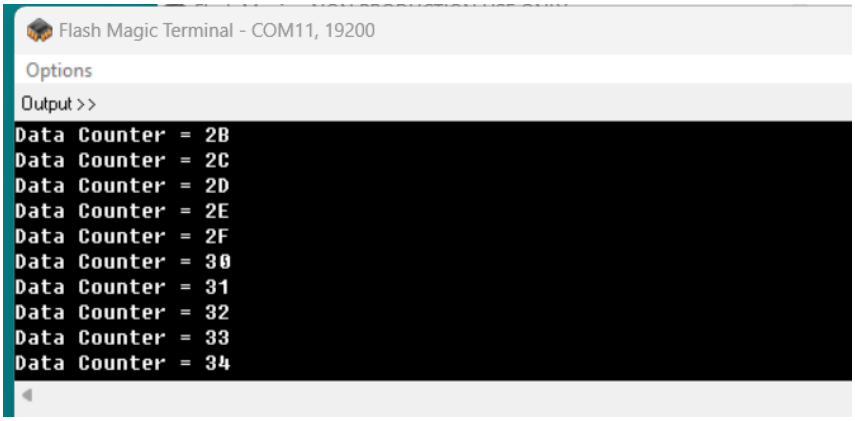
```
;============  Initial Serial Port  ============
Initial_Serial:     MOV         TMOD,#00100001B         ; T1M2 T0M1
                    MOV         SCON,#01010000B         ; Serial Mode1
                    MOV         TH1,#0FBH               ; 9600 FDH@18.432MHz
                    MOV         A,#00H                  ; FBH@18.432MHz
                    MOV         PCON,A                  ; SMOD = 0
                    CLR         EA                      ; Disable All
                    SETB        TR1                     ; Start Timer1
                    RET

;==========  Send Data to Serial Port  ==========
SEND_2HEX:          PUSH        Acc
                    SWAP        A
                    CALL        SEND_1HEX
                    POP         Acc
SEND_1HEX:          PUSH        DPH
                    PUSH        DPL
                    MOV         DPTR,#HEXASC_TAB
                    ANL         A,#0FH
                    MOVC        A,@A+DPTR
                    POP         DPL
                    POP         DPH
SEND_ASCII:         PUSH        IE
                    CLR         ES
                    CLR         TI
                    MOV         SBUF,A
                    JNB         TI,$
                    CLR         TI
                    POP         IE
                    RET
HEXASC_TAB:         DB          '0123456789ABCDEF'

;=========  Send Line Feed ==================
Send_LineFeed:      MOV         A,#0DH
                    CALL        Send_ASCII
                    MOV         A,#0AH
                    CALL        Send_ASCII
                    RET

;========  Send Line Feed ==================
Send_ClrScr:        MOV         A,#0CH
                    CALL        Send_ASCII
                    RET

;== Send Data Table to Serial Start@DPTR to '00H' =
Send_Table:         CLR         A
                    MOVC        A,@A+DPTR
                    JZ          _X_Send_Tab
                    CALL        SEND_ASCII
                    INC         DPTR
                    JMP         Send_Table
_X_Send_Tab:        RET

;========= Send 1-Byte@Acc to SERIAL =========
TX_BYTE:            PUSH        IE
                    CLR         ES
                    CLR         TI
                    MOV         SBUF,A
                    JNB         TI,$
                    CLR         TI
                    POP         IE
                    RET

;====== Receive Data From SERIAL to Acc ========
RX_BYTE:            PUSH        IE
                    CLR         ES
                    JNB         RI,$                    ; Wait data
                    CLR         RI
                    MOV         A,SBUF
                    POP         IE
                    RET
```

; 2/3 Edit and Save to "myDelay.A51"

```
;##############################################
;#*********************************************
;#     All Delay for XTAL 18.432MHz 12Clk/MC
;#**********************************************
DELAY_4SEC:     PUSH    07H                     ; Delay 80*50MilliSec
                MOV     R7,#80                  ;  = 4000MilliSec
                JMP     _DELAY_50M

DELAY_2SEC:     PUSH    07H                     ; Delay 40*50MilliSec
                MOV     R7,#40                  ;  = 2000MilliSec
                JMP     _DELAY_50M

DELAY_1SEC:     PUSH    07H                     ; Delay 20*50MilliSec
                MOV     R7,#20                  ;  = 1000MilliSec
                JMP     _DELAY_50M

DELAY_500M:     PUSH    07H                     ; Delay 10*50MilliSec
                MOV     R7,#10                  ;  = 500MilliSec
                JMP     _DELAY_50M

DELAY_250M:     PUSH    07H                     ; Delay 5*50 MilliSec
                MOV     R7,#5                   ;  = 250 MilliSec
                JMP     _DELAY_50M

DELAY_100M:     PUSH    07H                     ; Delay 2*50 MilliSec
                MOV     R7,#2                   ;  = 100 MilliSec
                JMP     _DELAY_50M

DELAY_50M:      PUSH    07H                     ; Delay 1*50 MilliSec
                MOV     R7,#1                   ;  = 50 MilliSec
                JMP     _DELAY_50M


_DELAY_50M:     PUSH    Acc
                PUSH    B
_DELAY_050M:    MOV     B,#200                  ; Delay 50 MilliSec
_DELAY_250U:    MOV     A,#191                  ; 1MC ,250 uSec
                DJNZ    Acc,$                   ; 2MC ,(192-1)*0.651*2 = 250 uSec
                DJNZ    B,_DELAY_250U           ; 250*200=50mS
                DJNZ    R7,_DELAY_050M          ; 50*5 =250mS
                POP     B
                POP     Acc
                POP     07H
                RET
```

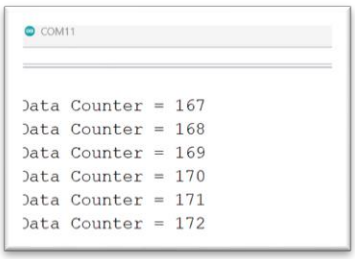
3/3 Edit and Save to "Lab10Srl3.ASM"

```
T2CON           EQU     0C8H
RCAP2L          EQU     0CAH
RCAP2H          EQU     0CBH

Counter         EQU     40H                     ; Lab10Srl3.Sub
PortO_LEDGreen  EQU     P3.7

                ORG     0000H
                JMP     0100H

                ORG     0100H
                MOV     SP,#3FH
                CLR     EA
                CALL    Delay_250M
                CALL    Initial_Serial
                CALL    Send_ClrScr
                MOV     Counter,#00
Main_Loop:      MOV     DPTR,#Header_Fram
                CALL    Send_Table
                MOV     A,Counter
                CALL    Send_2HEX
                CALL    Send_LineFeed
                CPL     PortO_LEDGreen
                INC     Counter
                CALL    Delay_500M
                JMP     Main_Loop

$INCLUDE (myDelay.A51)
$INCLUDE (mySerial.A51)

Header_Fram:    DB      'Data Counter = ',00
                END
```



RideC51 >> "Lab10Srl4.C"

```
#include <reg52.h>
#include <stdio.h>

sbit LED_0 = P3 ^ 6;
sbit LED_1 = P3 ^ 7;
sbit CS_Seg0 = P1 ^ 4;
sbit CS_Seg1 = P1 ^ 5;
#define PortO_DispData P0
unsigned int Count, Counter = 0;
char myString[80];
code unsigned char CodeSeg[] =
{ 0x77, 0x11, 0x6D, 0x5D, 0x1B, 0x5E, 0x7E, 0x15,
  0x7F, 0x5F, 0x3F, 0x7A, 0x66, 0x79, 0x6E, 0x2E
};

void Delay(unsigned char msec)
{ unsigned char i, j;
  while (msec)
  { for (i = 1; i <= 250; i++) //not :
      for (j = 1; j <= 113; j++); //with ;
    msec--;
  }
}

void Timer0_ISR(void) interrupt 1
{ unsigned char Temp;
  if (Count % 2 == 0) {
    Temp = Counter & 0x0f;
    CS_Seg0 = 1;     CS_Seg1 = 0;
  }
  else {
    Temp = (Counter >> 4) & 0x0f;
    CS_Seg0 = 0;     CS_Seg1 = 1;
  }
  PortO_DispData = ~CodeSeg[Temp];
  Count++;
}

void uart_tx(unsigned char serialdata) {
  SBUF = serialdata; //Load Data to Serial Buffer Register
  while (TI == 0); //Wait Until Transmission To Complete
  TI = 0; //Clear Transmission Interrupt Flag
}

void uart_msg(unsigned char * c) {
  while ( * c != 0)  {
    uart_tx( * c++);
  }
}

//=====================================
main()
{ TMOD = 0x22;  // Time1Mode2 Timer0Mode1

  SCON = 0x50;
  TH1 = 0xFB;   // Baud=9600 @18.432MHz x1
  TR1 = 1;

  TH0 = 0;
  TR0 = 1;
  ET0 = 1; // Enable External 0 Interrupt
  EA = 1;  // Global Interrupt Enable
  while (1)
  { Delay(50);
    Counter++;
    sprintf(myString, "\nData Counter = %d", Counter);
    uart_msg(myString);
    LED_0 = (Counter + 1) % 2;
    LED_1 = Counter % 2;
  }
}
```



Interrupt	Name	Numbers
External Interrupt 0	(INT0)	0
Timer Interrupt 0	(TF0)	1
External Interrupt 1	(INT1)	2
Timer Interrupt 1	(TF1)	3
Serial Communication	(RI + TI)	4
Timer 2 (8052 only)	(TF2)	5

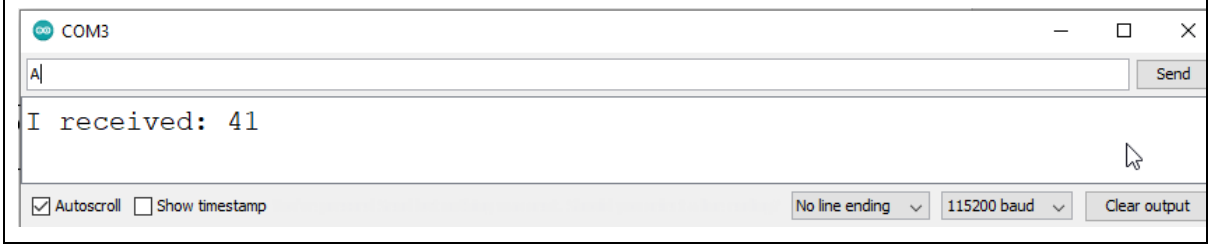
Edit and Save to "Lab10Srl5.ASM" Receive via Serial Interrupt

```
T2CON       EQU     0C8H
RCAP2L      EQU     0CAH
RCAP2H      EQU     0CBH
RX_Data     EQU     30H

            ORG     0000H
            JMP     0100H

            ORG     0023H
            JMP     Serial_ISR

            ORG     0100H
            MOV     SP,#3FH
            CLR     EA
            CALL    Delay_250M
            CALL    Initial_Serial
            CALL    Send_ClrScr
            SETB    ES
            SETB    EA

Main_Loop:  CALL    Display
            JMP     Main_Loop

Display:    MOV     DPTR,#Seg_Tab
            MOV     A,RX_Data
            SETB    P1.4
            CLR     P1.5
            CALL    _Display
            MOV     A,RX_Data
            CLR     P1.4
            SETB    P1.5
            SWAP    A
_Display:   ANL     A,#0FH
            MOVC    A,@A+DPTR
            CPL     A
            MOV     P0,A
            CLR     A
            DJNZ    Acc,$
            RET

Seg_Tab:    DB      77H, 11H, 6DH, 5DH, 1BH, 5EH, 7EH, 15H
            DB      7FH, 5FH, 3FH, 7AH, 66H, 79H, 6EH, 2EH

Serial_ISR: CLR     ES              ; Disable Serial Interrupts
            JB      TI,_X_Srl       ; If Tx Exit
            MOV     A,SBUF          ; Save Data
            MOV     SBUF,A          ; Echo Data
            MOV     Rx_Data,A       ; Save to Variable
_X_Srl:     CLR     TI              ; Clear Ti Flag
            CLR     RI              ; Clear Ri Flag
            SETB    ES              ; Enable Serial Interrupts
            RETI                    ; Return from Interrupts

$INCLUDE (myDelay.A51)
$INCLUDE (mySerial.A51)

            END
```

4/8: ESP32 Serial Communication

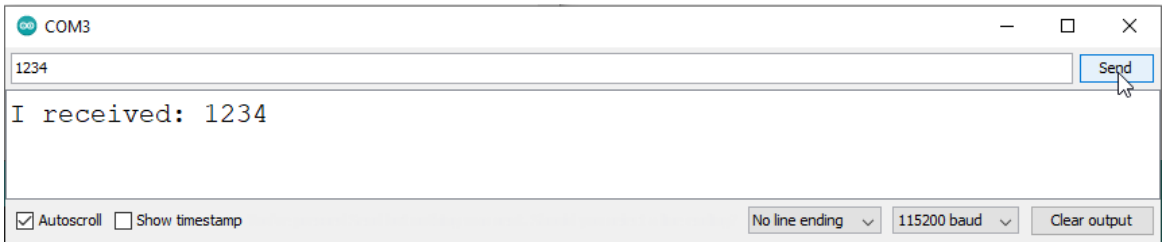
Test21 - Get Data from Serial Port with Serial Read

```
int incomingByte = 0; // for incoming serial data

void setup() {
  Serial.begin(115200); // opens serial port, sets data rate to 115200 bps
}

void loop() {
  // reply only when you receive data:
  if (Serial.available() > 0) {
    // read the incoming byte:
    incomingByte = Serial.read();

    // say what you got:
    Serial.print("I received: ");
    Serial.println(incomingByte, HEX);
  }
}
```



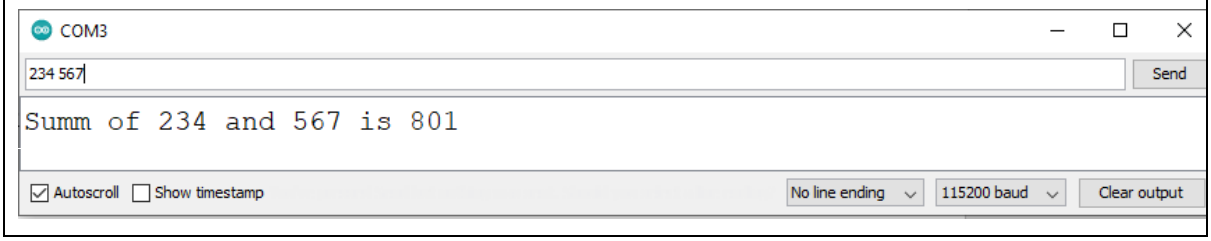
Test22 - Get Data from Serial Port with Serial parseInt

```
int incomingData = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  if (Serial.available() > 0) {
    incomingData = Serial.parseInt();
    Serial.print("I received: ");
    Serial.println(incomingData);
  }
}
```



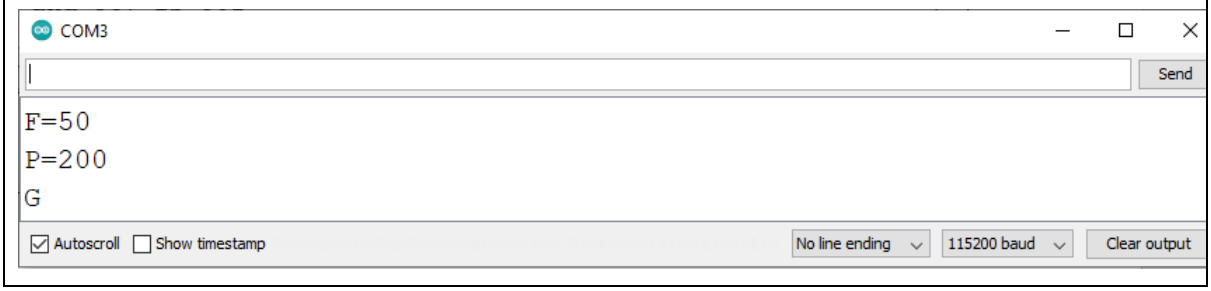
Test23 - Get 2Data from Serial Port with Serial parseInt ➔ separate number with space

```
int Data1, Data2;
char buffer[40];

void setup() {
 Serial.begin(115200);
}

void loop() {
 if (Serial.available() > 0) {
   Data1 = Serial.parseInt();
   Data2 = Serial.parseInt();
   sprintf(buffer, "Summ of %d and %d is %d", Data1, Data2, Data1 + Data2);
   Serial.println(buffer);
 }
}
```



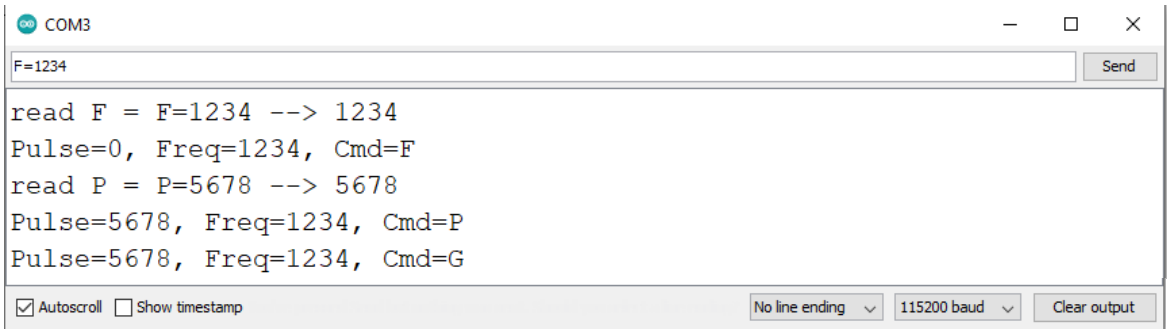
Test24 - Read String

```
String strData;
int Data1, Data2;

void setup() {
 Serial.begin(115200);
}

void loop() {
 if (Serial.available() > 0) {
   strData = Serial.readString();
   Serial.println(strData);
 }
}
```



Test25 - Read String and Split ➔ P=xxx, F=yyy, G

```
String Cmd, strData;
int strInt, strlength, Pulse = 0, Freq = 0;
char buffer[40];

void setup() {
 Serial.begin(115200);
}

void loop() {
 if (Serial.available() > 0) {
  strData = Serial.readString();
  strlength = strData.length();
  Cmd = strData.substring(0, 1);
  strInt = strData.substring(2, strlength).toInt();

  if (Cmd == "F") {
   if (strInt != 0) Freq = strInt;
   sprintf(buffer, "read F = %S --> %d", strData, strInt);
   Serial.println(buffer);
  }

  if (Cmd == "P") {
   if (strInt != 0) Pulse = strInt;
   sprintf(buffer, "read P = %S --> %d", strData, strInt);
   Serial.println(buffer);
  }

  sprintf(buffer, "Pulse=%d, Freq=%d, Cmd=%s", Pulse, Freq, Cmd);
  Serial.println(buffer);
 }
}
```



Question-ESP32-01: เขียนโปรแกรมสร้างสัญญาน โดยมีข้อกำหนดดังนี้

- รับคำสั่งเป็น Serial Command จำนวน 4 คำสั่ง คือ P=xxx, D=yy, G, ? โดย
  - P=xxx<Cr>
    - ต้องมี Carriage return
    - กำหนดจำนวน Pulse ให้มีค่าระหว่าง 100-999
    - หากค่านี้เป็น 0 ให้ทำงานต่อเนื่องตลอดเวลา
  - D=yy<Cr>
    - ต้องมี Carriage return
    - กำหนดค่า Delay ให้มีค่าระหว่าง 10-99 หน่วยเป็น มิลลิวินาที
    - เมื่อป้อนแล้วให้ Echo แจ้งด้วยว่าจะได้ความถี่เท่าใด
  - G
    - ไม่ต้องมี Carriage return
    - ให้ ESP32 เริ่มทำงาน
  - ?
    - ไม่ต้องมี Carriage return
    - ให้ Echo แจ้งว่าใครเป็นคนเขียน โปรแกรมเวอร์ชันเท่าไหร่ คอมมานอะไรบ้าง

- สร้างสัญญาน Pulse ตามคำสั่งออกที่ D23
- เมื่อทำงานเสร็จให้ D23 = High
- ระหว่าง ESP32 ทำงานสามารถส่ง Serial Command ให้เปลี่ยนค่าดีเลย์ จำนวนพัลส์ได้

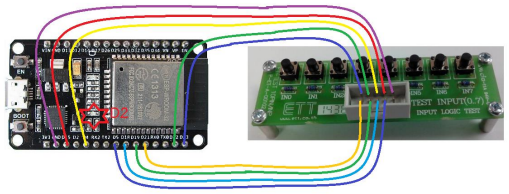
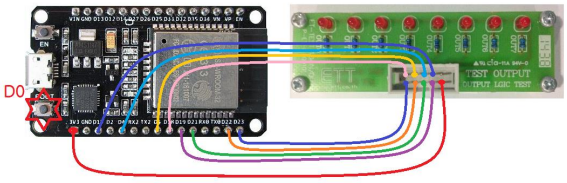
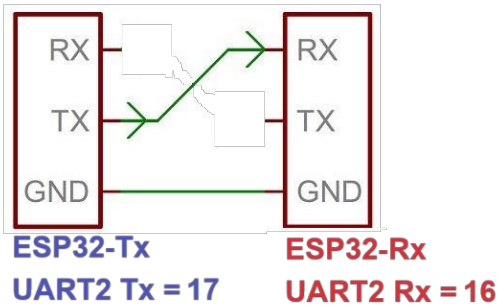
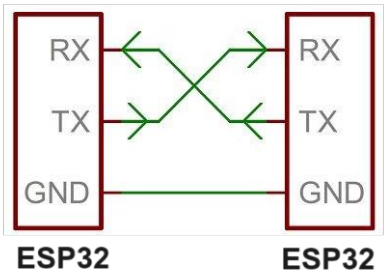
Question-ESP32-02: เขียนโปรแกรมส่ง-รับข้อมูล โดยมีข้อกำหนดดังนี้

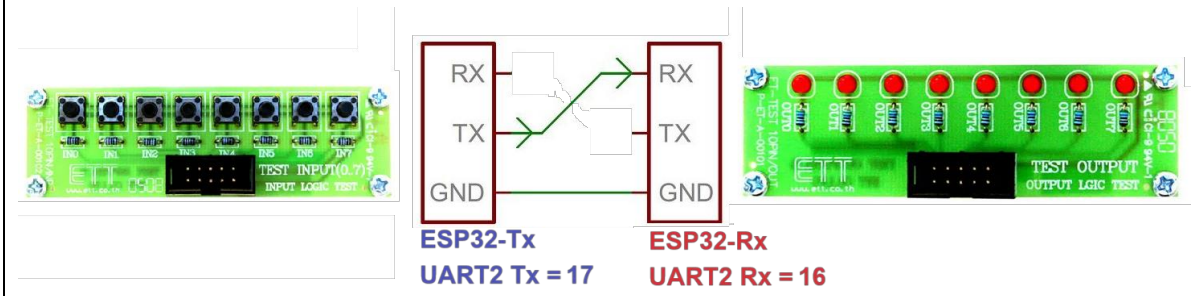
TX Board: ให้เป็นบอร์ดอ่านค่าสวิตซ์แล้วส่งข้อมูลออก Serial

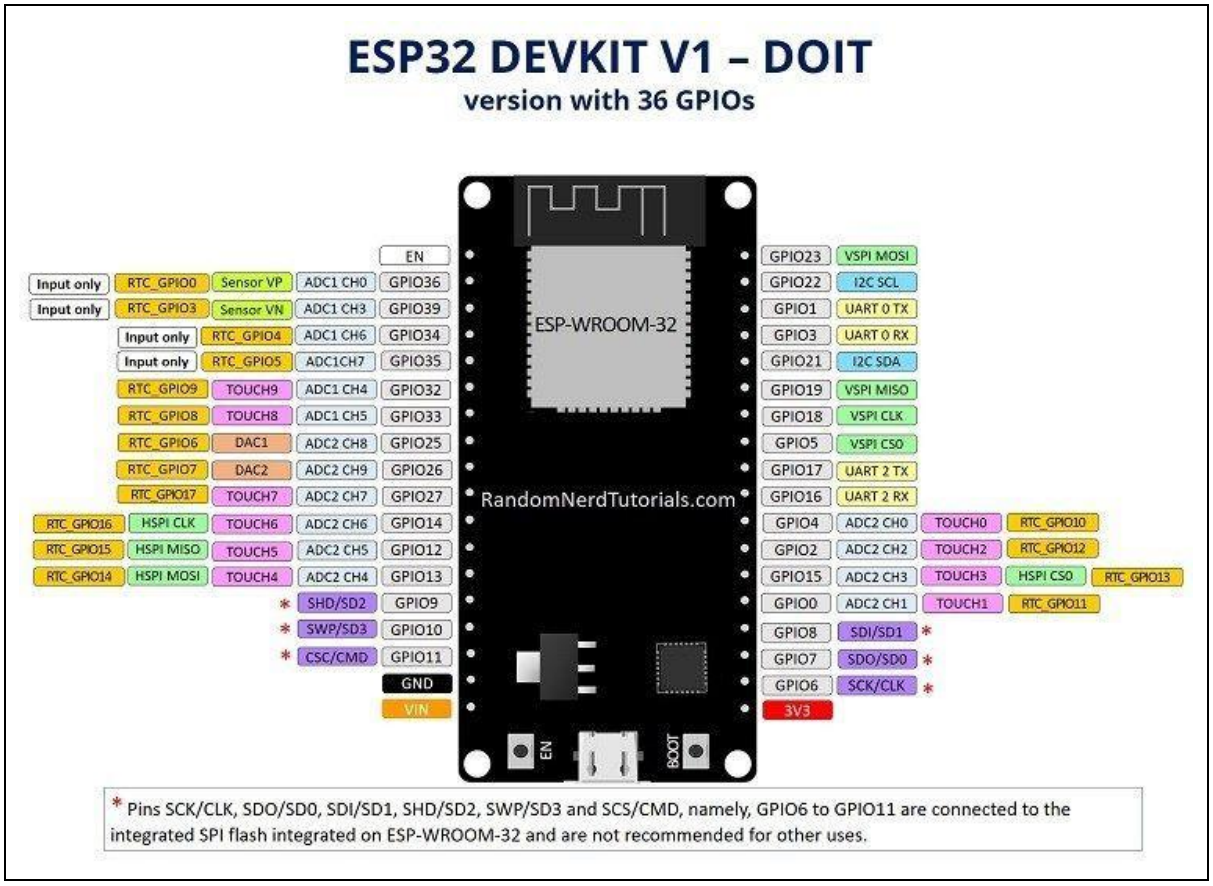
- ต่อวงจรตาม “Digital Input – 8 Sw”, ใช้ Serial2 ในการส่งข้อมูล
- วนรอบตรวจการกดสวิตซ์ที่ต่อกับ ESP32 Rx Board จำนวน 8 ตัว
- หากมีการกดสวิตซ์ใดให้ ส่งรหัส ASCII เลขสวิตซ์ไปยังบอร์ด RX พร้อมทั้งแสดงเลขสวิตซ์ที่ Serial Terminal (แว่นขยาย) ของบอร์ดส่ง

RX Board: ให้เป็นบอร์ดรับค่าจาก Serial แล้วแสดงที่ LED

- ต่อวงจรตาม “Digital Output – 8 LED”, ใช้ Serial2 ในการรับข้อมูล
- วนรอบรอรับค่าจาก ESP-32 Tx Board
- นำค่ารหัส ASCII ที่รับได้แสดงผลที่ Serial Terminal (แว่นขยาย) ของบอร์ดรับ และ 8 LED Board

Tx-Board	Rx-Board
	
Tx-Rx System – One Way	Tx-Rx System – Two Way
	




ESP32 DEVKIT V1 – DOIT
version with 36 GPIOs
ESP-WROOM-32
RandomNerdTutorials.com
EN
BOOT
EN
Input only RTC_GPIO0 Sensor VP ADC1 CH0 GPIO36
Input only RTC_GPIO3 Sensor VN ADC1 CH3 GPIO39
Input only RTC_GPIO4 ADC1 CH6 GPIO34
Input only RTC_GPIO5 ADC1CH7 GPIO35
RTC_GPIO9 TOUCH9 ADC1 CH4 GPIO32
RTC_GPIO8 TOUCH8 ADC1 CH5 GPIO33
RTC_GPIO6 DAC1 ADC2 CH8 GPIO25
RTC_GPIO7 DAC2 ADC2 CH9 GPIO26
RTC_GPIO17 TOUCH7 ADC2 CH7 GPIO27
RTC_GPIO16 HSPI CLK TOUCH6 ADC2 CH6 GPIO14
RTC_GPIO15 HSPI MISO TOUCH5 ADC2 CH5 GPIO12
RTC_GPIO14 HSPI MOSI TOUCH4 ADC2 CH4 GPIO13
* SHD/SD2 GPIO9
* SWP/SD3 GPIO10
* CSC/CMD GPIO11
GND
VIN
GPIO23 VSPI MOSI
GPIO22 I2C SCL
GPIO1 UART 0 TX
GPIO3 UART 0 RX
GPIO21 I2C SDA
GPIO19 VSPI MISO
GPIO18 VSPI CLK
GPIO5 VSPI CS0
GPIO17 UART 2 TX
GPIO16 UART 2 RX
GPIO4 ADC2 CH0 TOUCH0 RTC_GPIO10
GPIO2 ADC2 CH2 TOUCH2 RTC_GPIO12
GPIO15 ADC2 CH3 TOUCH3 HSPI CS0 RTC_GPIO13
GPIO0 ADC2 CH1 TOUCH1 RTC_GPIO11
GPIO8 SDI/SD1 *
GPIO7 SDO/SD0 *
GPIO6 SCK/CLK *
3V3
* Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and SCS/CMD, namely, GPIO6 to GPIO11 are connected to the integrated SPI flash integrated on ESP-WROOM-32 and are not recommended for other uses.

5/8: ESP32 OTA (Over the Air) Programming

- https://lastminuteengineers.com/esp32-ota-updates-arduino-ide/
- https://lastminuteengineers.com/esp32-ota-web-updater-arduino-ide/
- https://randomnerdtutorials.com/esp32-over-the-air-ota-programming/
- https://randomnerdtutorials.com/esp32-over-the-air-ota-programming/

Ways To Implement OTA In ESP32

There are two ways to implement OTA functionality in ESP32.

- Basic OTA – Over-the-air updates are sent through Arduino IDE.
- Web Updater OTA – Over-the-air updates are sent through a web browser.

Test31 – Basic OTA

- https://lastminuteengineers.com/esp32-ota-updates-arduino-ide/

Test32 – Web Updater OTA

- https://lastminuteengineers.com/esp32-ota-web-updater-arduino-ide/

Test33 – IAP (In Application Programing)

- https://randomnerdtutorials.com/esp32-over-the-air-ota-programming/

Test34 – Blynk OTA

- https://docs.blynk.io/en/blynk.edgent/updating-devices-firmwares-ota

ทดลอง Upload Code สลับกันไปมาระหว่าง Blink01 กับ Blink02 ด้วยวิธีการทั้ง 4 วิธี

Blink01	Blink02

`void setup() {`
`  pinMode(LED_BUILTIN, OUTPUT);`
`}`
 	
`void loop() {`
`  digitalWrite(LED_BUILTIN, HIGH);`
`  delay(500);`
`  digitalWrite(LED_BUILTIN, LOW);`
`  delay(500);`
`}`
	`void setup() {`
	`  pinMode(LED_BUILTIN, OUTPUT);`
	`}`
	
	`void loop() {`
	`  digitalWrite(LED_BUILTIN, HIGH);`
	`  delay(250);`
	`  digitalWrite(LED_BUILTIN, LOW);`
	`  delay(250);`
	`  digitalWrite(LED_BUILTIN, HIGH);`
	`  delay(250);`
	`  digitalWrite(LED_BUILTIN, LOW);`
	`  delay(1000);`
	`}`

6/8: Industrial Communication

Modbus: Industrial Serial Protocol

6.1 ETT-RS485 v2 Board - Input/Output and MQTT

6.2 ETT-RS485 v2 Board – Blynk2.0

6.3 RS485 MODBUS/RTU Read Modbus Device

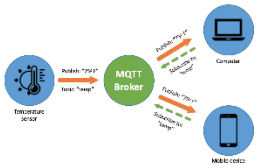
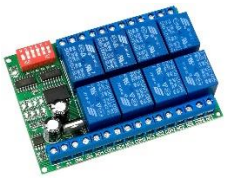
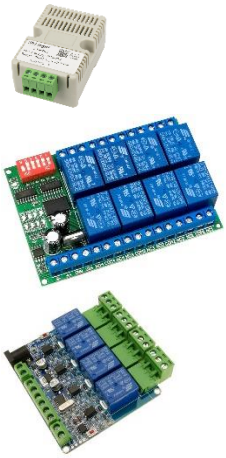
6.4 RS485 MODBUS/RTU Write Modbus Device

6.5 RS485 MODBUS/RTU Read/Write Device

6.6 Blynk2.0 to MODBUS/RTU Device

6.7 MODBUS/TCP Client Mode -- Read/Write Device

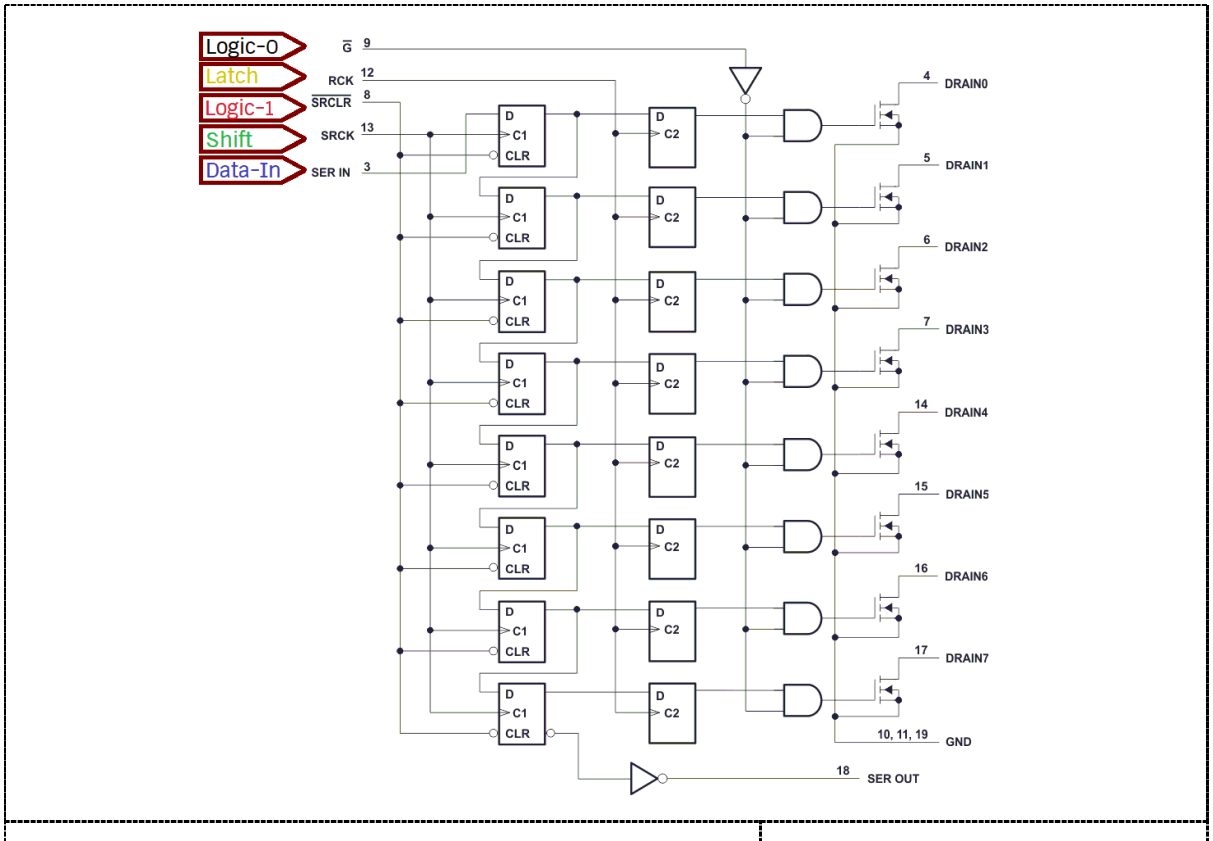
6.8 MODBUS/TCP Server Mode

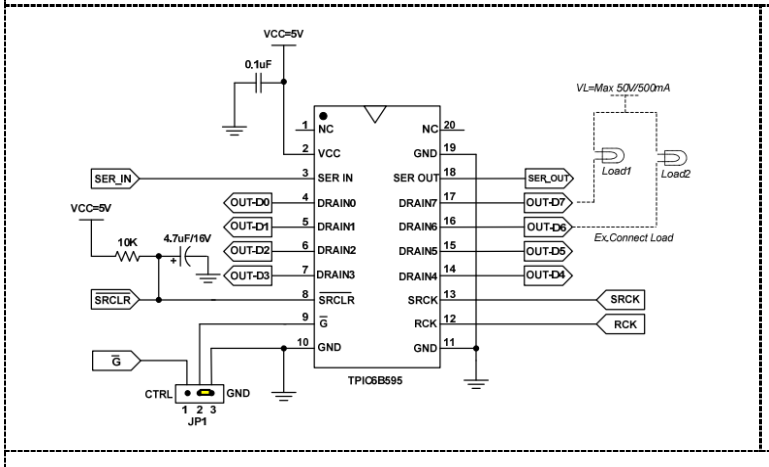
IoT with MQTT Lens		Blynk 2.0	
			
Read Modbus Device		**Write Modbus Device**	
			
Read/Write Modbus Device		**Blynk 2.0 ↔ Modbus Device**	
			

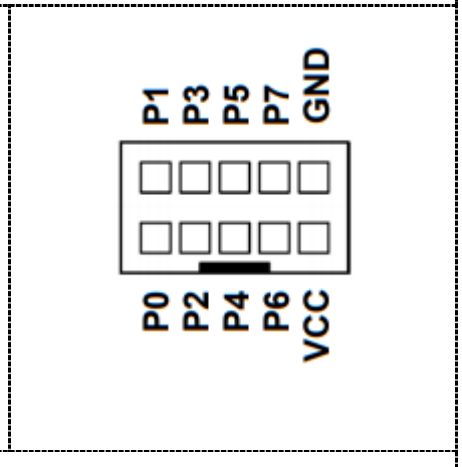
< Insert -- Modbus: Industrial Serial Protocol >

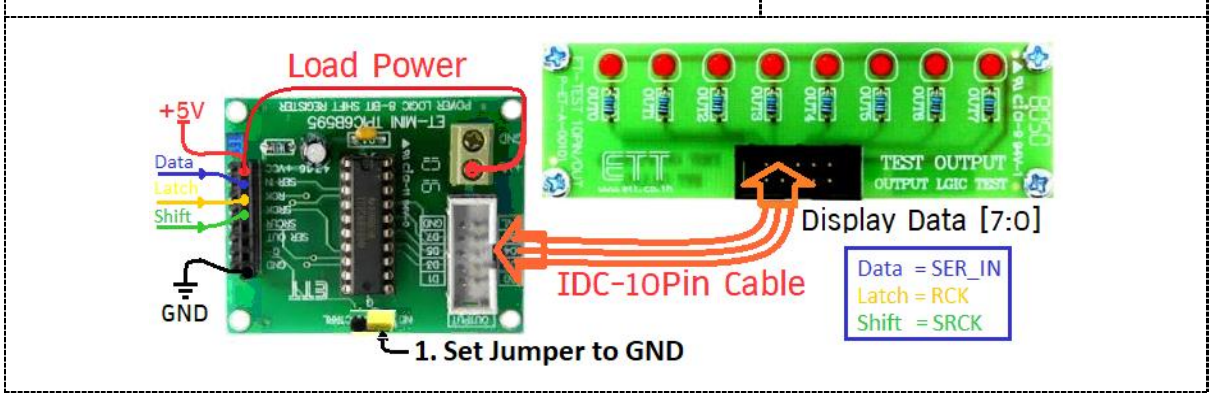
7/8: Hardware

1. Shift Output 8Bit ด้วย TPIC6B595 – Power Logic 8-bit Shift register

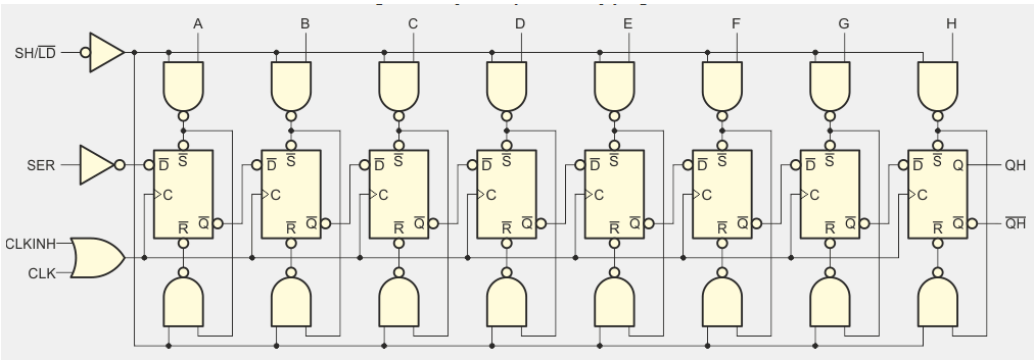


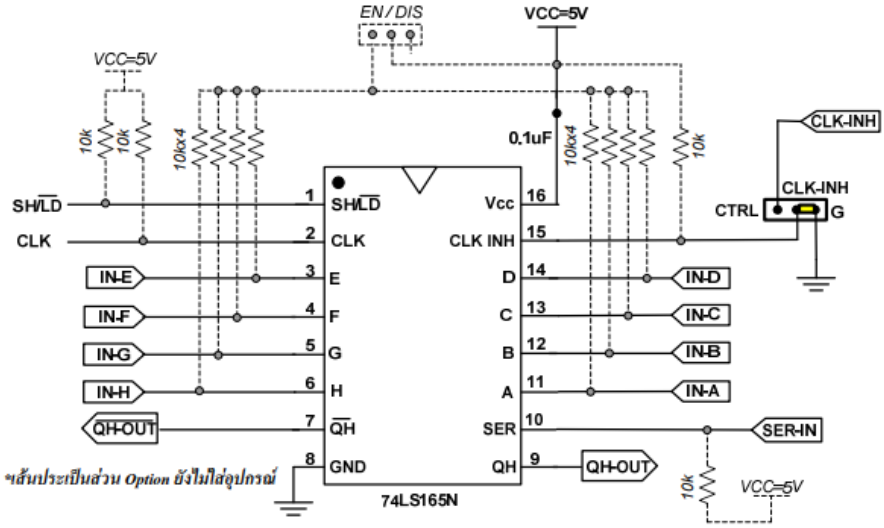


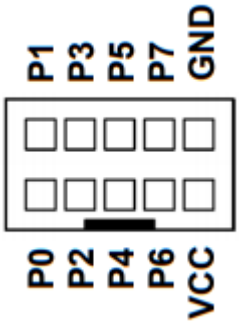


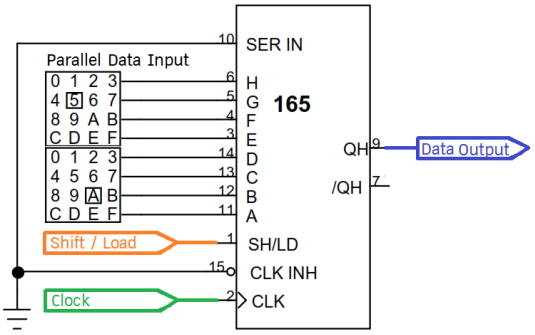


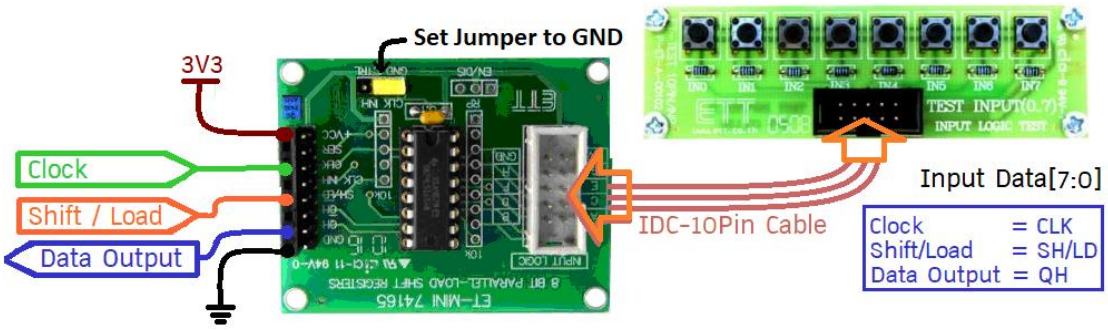
2. Shift Input 8Bit: 74165 – Parallel Load Shift Register



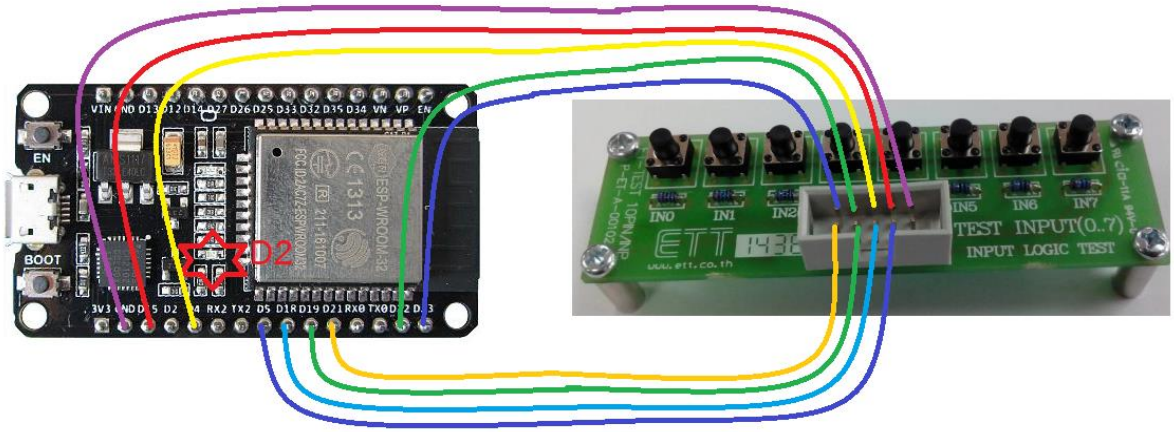


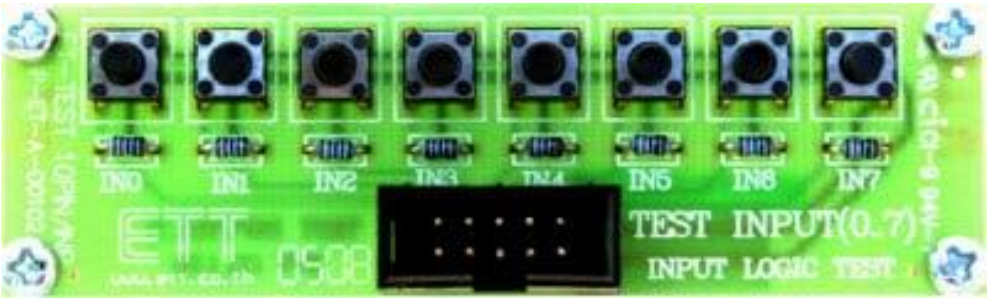


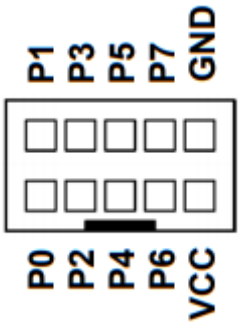


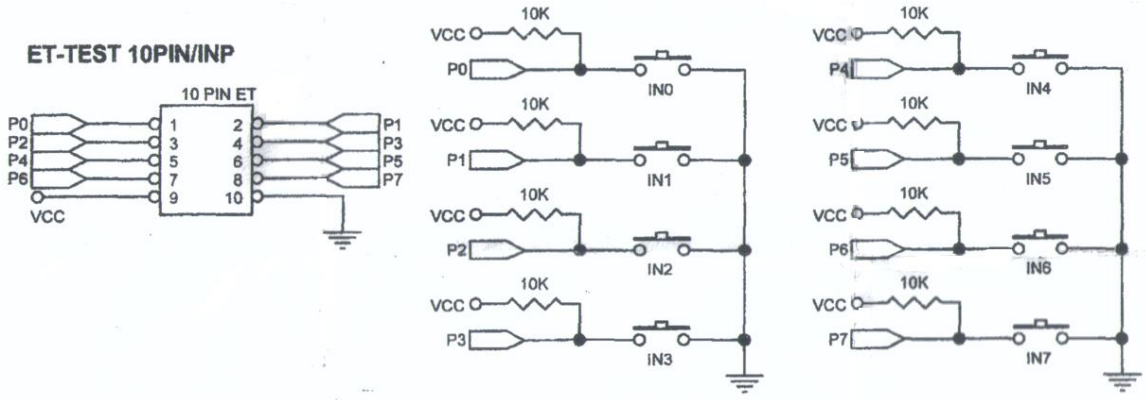


3. Digital Input – 8 SW

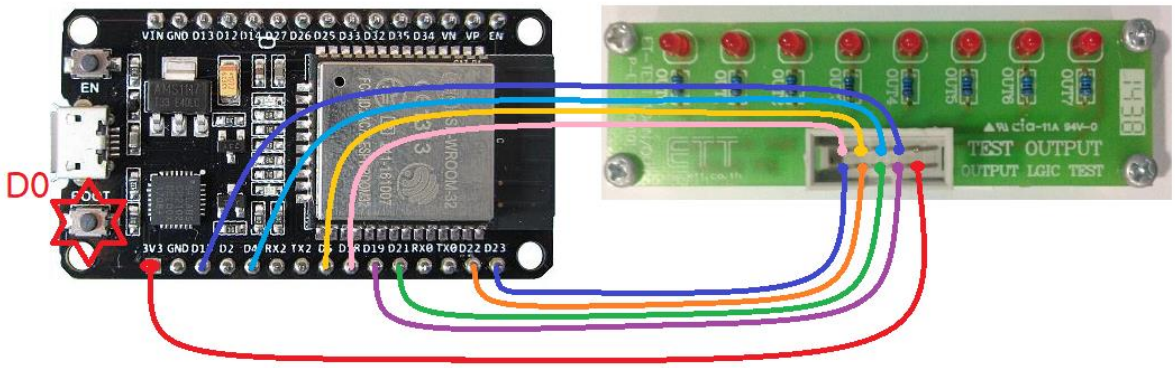


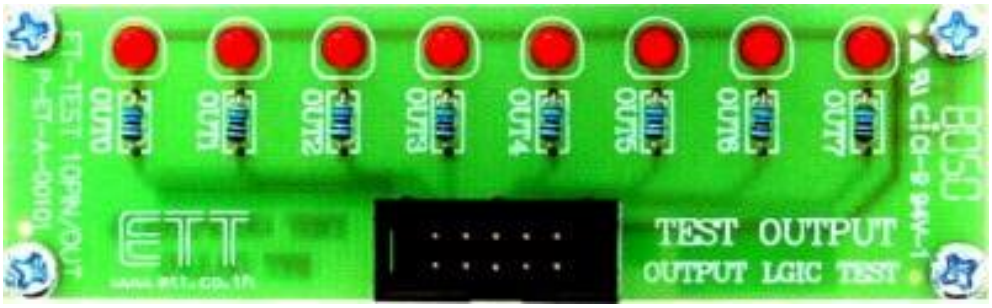


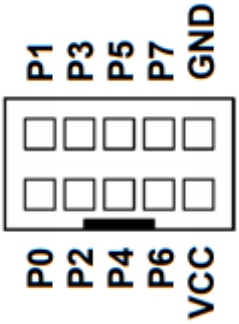


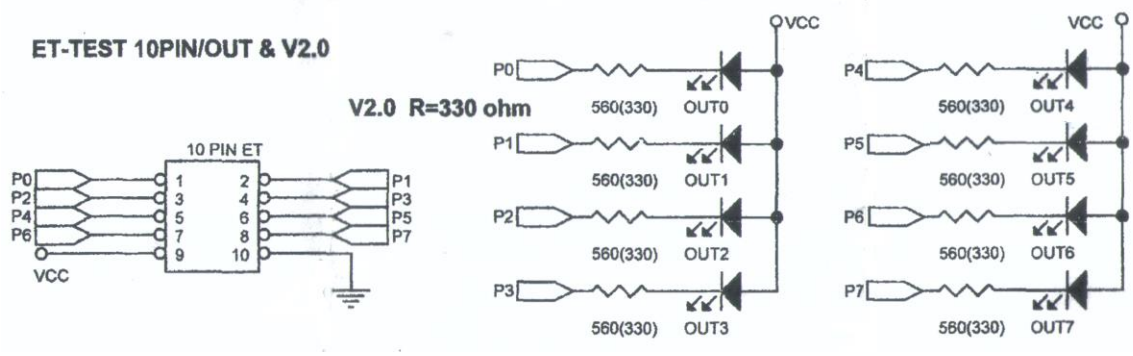


4. Digital Output – 8 LED

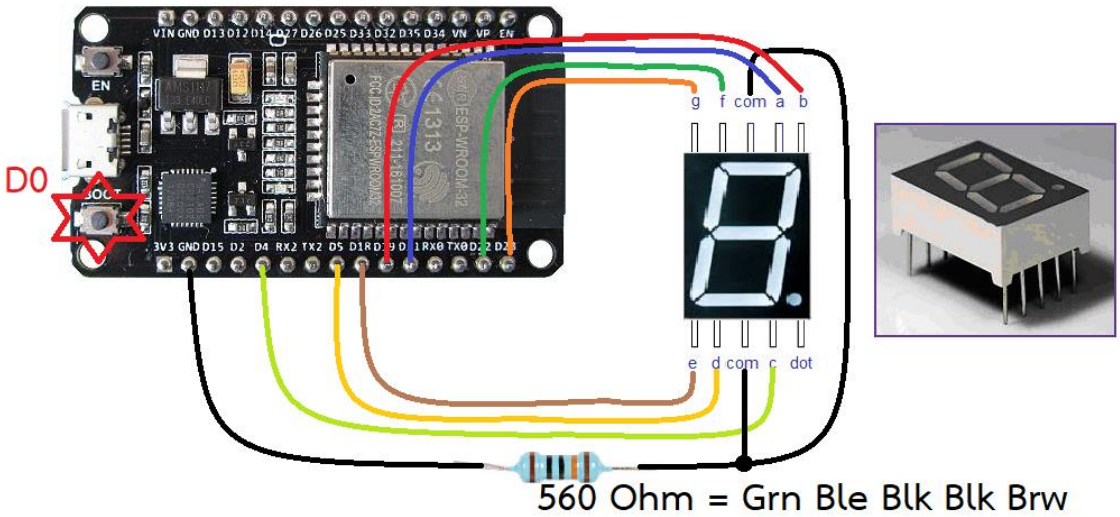


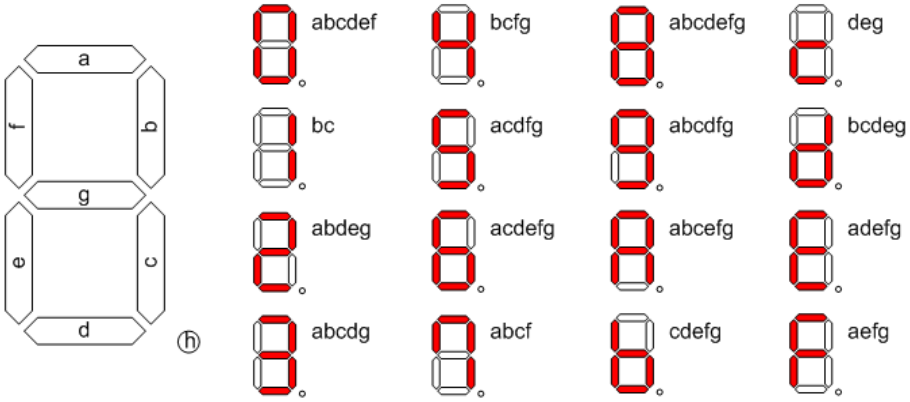


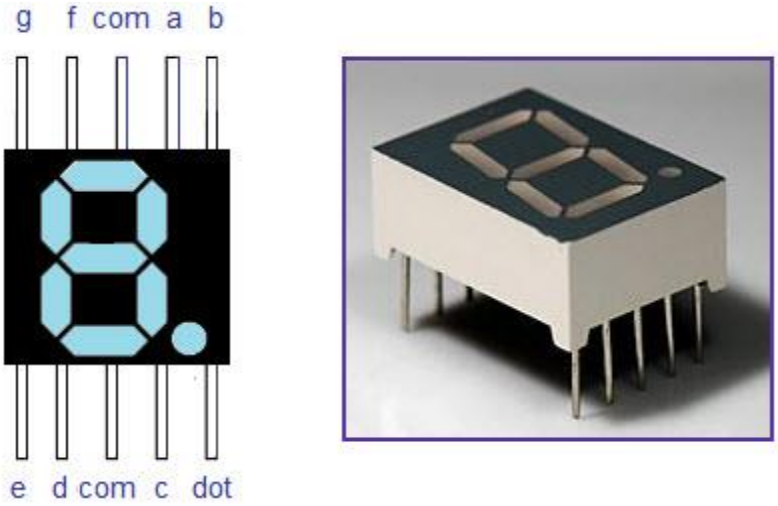




5. Digital Output – 7_Segment





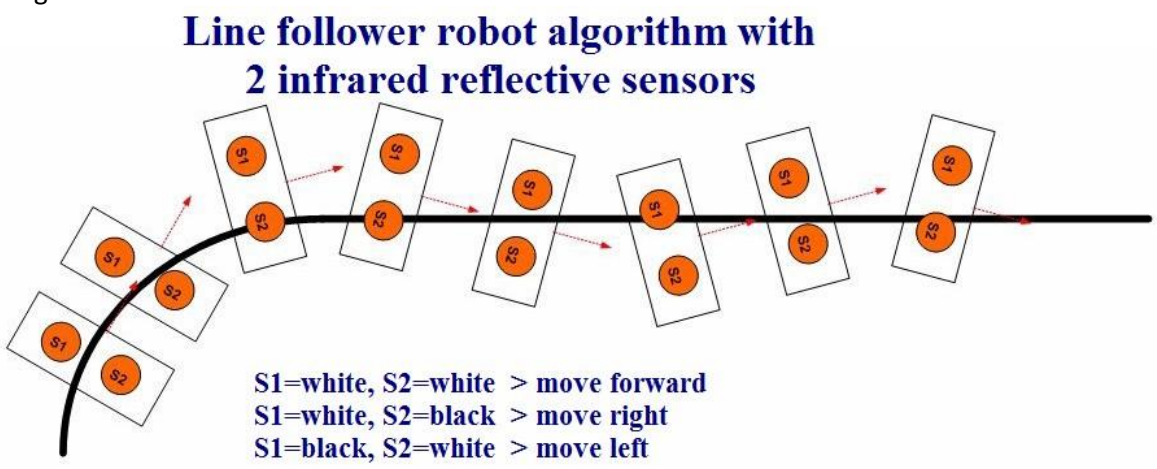
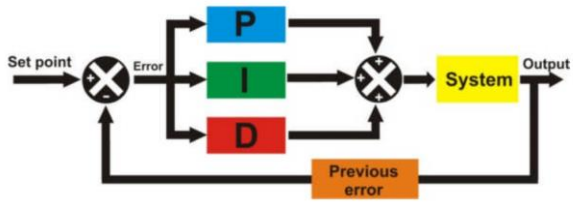
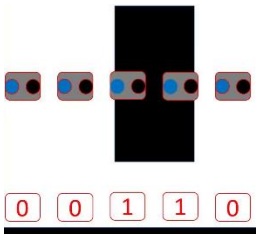
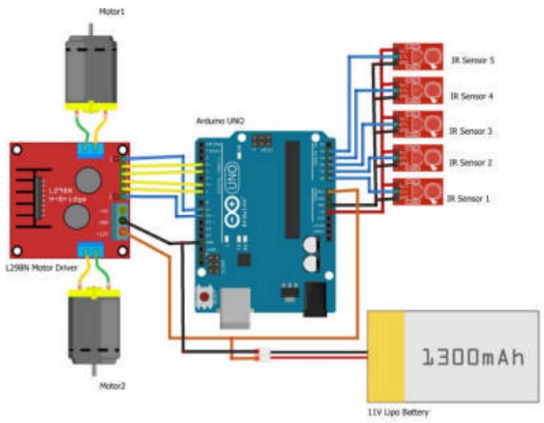
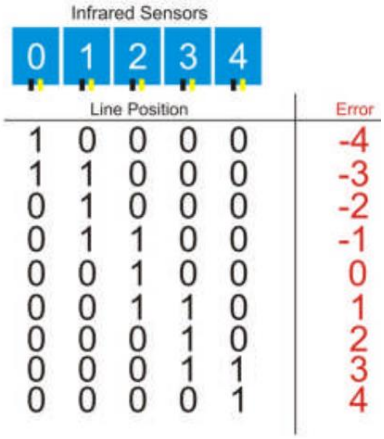


8/8: Line Follower Robot (2)

Arduino Five Sensor PID Line Following Robot

- https://www.hackster.io/mjrobot/line-follower-robot-pid-control-android-setup-e5113a
- https://www.hackster.io/anova9347/line-follower-robot-with-pid-controller-cdedbd
- https://projecthub.arduino.cc/anova9347/line-follower-robot-with-pid-controller-01813f
- https://robotsiam.blogspot.com/2016/10/pid-control-arduino-line-follower-robot.html
- https://youtu.be/flvoxn0BbCs
- https://youtu.be/ZvOUs0dBGPI
- https://eprints.ums.ac.id/49936/1/Taufiq%20Journal%20FIX%20perpus.pdf
- https://www.engineersgarage.com/arduino-uno-driven-pid-line-follower/

Original



PID